Lab 9.1: Documentation Generation - Automatic Documentation and Code Comments

Name: Roger A Raju
Rollno: 2503a52370
Batch: 13

Task Description #1 (Documentation Formats - find_max)

Task: Consider the Python function find_max(numbers) and write documentation in docstring, inline-comment, and Google-style formats. Compare the three approaches and recommend the most effective style for a mathematical utilities library.

Prompt:

Use AI to document the find_max function in docstring, inline comments, and Google-style documentation. Include a critical comparison and a recommendation for a mathematical utilities library.

Original Code (Before Documentation):

```
def find_max(numbers):
    return max(numbers)
```

Documented Revised Version:

```
def find_max_docstring(numbers):
    """Return the largest number from a non-empty sequence.

    Args:
        numbers (list): A sequence of numeric values.

    Returns:
        float: The largest value in the sequence.

    Raises:
        ValueError: If the sequence is empty.
    """
    if not numbers:
        raise ValueError("numbers must not be empty")
    return max(numbers)


def find_max_inline(numbers):
    if not numbers:
        raise ValueError("numbers must not be empty")
    return max(numbers)


def find_max_google(numbers):
    """Find the maximum value in a sequence.

    Args:
```

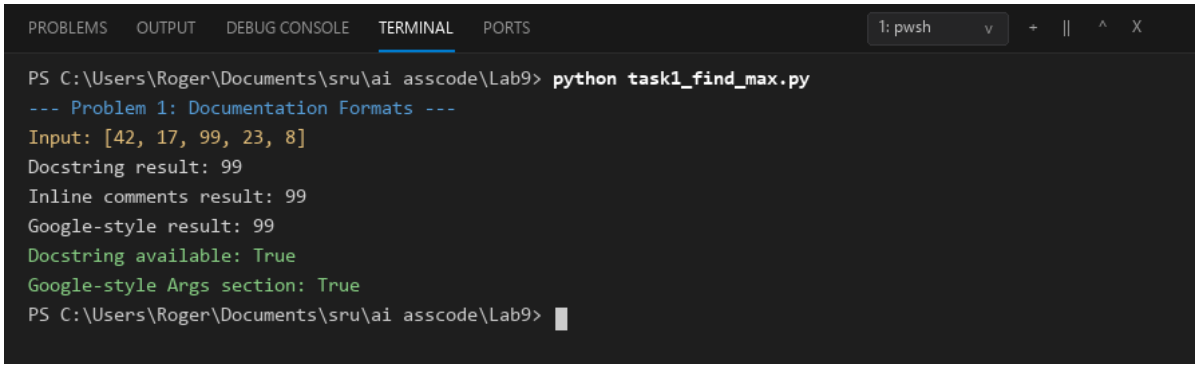
```
        numbers (list): A non-empty sequence of numbers.

    Returns:
        float: The largest number in the sequence.

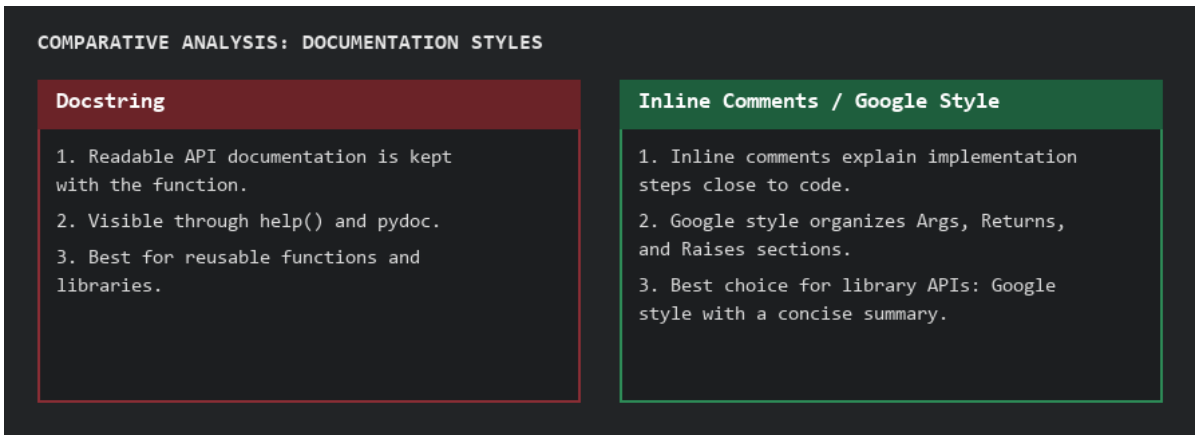
    Raises:
        ValueError: If numbers is empty.
    """
    if not numbers:
        raise ValueError("numbers must not be empty")
    return max(numbers)


if __name__ == "__main__":
    nums = [42, 17, 99, 23, 8]
    print("--- Problem 1: Documentation Formats ---")
    print(f"Input: {nums}")
    print(f"Docstring result: {find_max_docstring(nums)}")
    print(f"Inline comments result: {find_max_inline(nums)}")
    print(f"Google-style result: {find_max_google(nums)}")
    print(f"Docstring available: {bool(find_max_docstring.__doc__)}")
    print(f"Google-style Args section: {'Args:' in find_max_google.__doc__}")
```

Output:



Comparison:



Final Justification

All three documentation styles were implemented and demonstrated. For a mathematical utilities library, Google-style docstrings are recommended because they combine readable summaries with structured Args, Returns, and Raises sections that work well with pydoc and other documentation tools.

Task Description #2 (Documentation Formats - login)

Task: Consider the Python function login(user, password, credentials) and document it in all three formats. Compare the approaches and recommend the style most helpful for new developers onboarding a project.

Prompt:

Use AI to document the login function in docstring, inline comments, and Google-style documentation. Compare the approaches and recommend the best style for developer onboarding.

Original Code (Before Documentation):

```
def login(user, password, credentials):
    return credentials.get(user) == password
```

Documented Revised Version:

```
def login_docstring(user, password, credentials):
    """Validate a user's password against a credential mapping.

    Args:
        user (str): Username to authenticate.
        password (str): Password supplied by the user.
        credentials (dict): Mapping of usernames to passwords.

    Returns:
        bool: True when the supplied password matches.
    """
    return credentials.get(user) == password


def login_inline(user, password, credentials):
    stored_password = credentials.get(user)
    return stored_password == password


def login_google(user, password, credentials):
    """Authenticate a user using stored credentials.

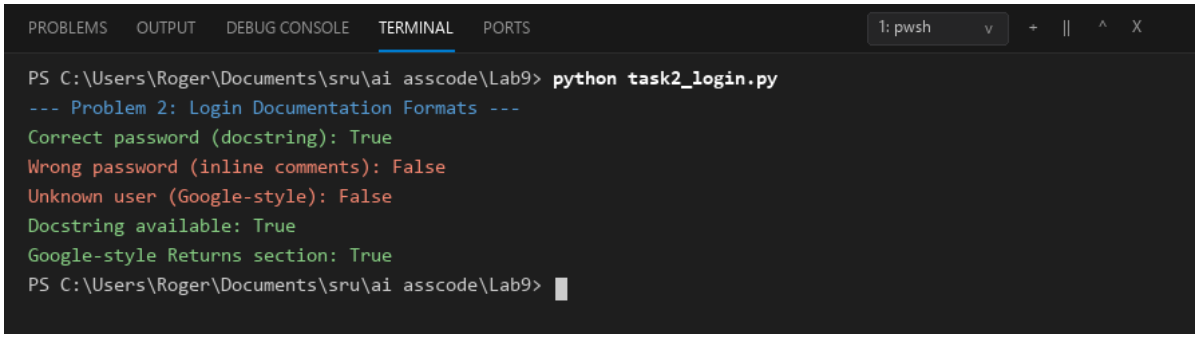
    Args:
        user (str): Username to check.
        password (str): Password provided during login.
        credentials (dict): Username-to-password mapping.

    Returns:
        bool: Authentication result.
    """
    return credentials.get(user) == password

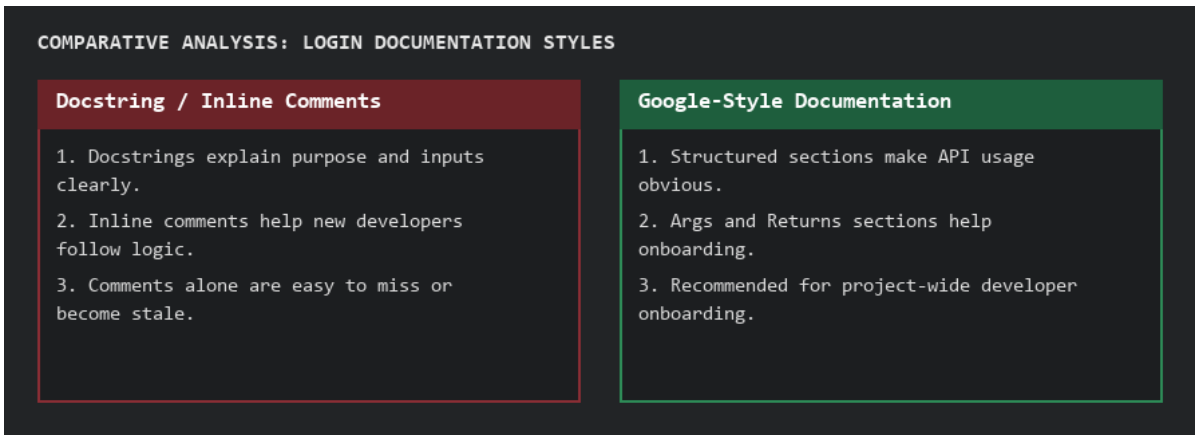

if __name__ == "__main__":
    creds = {"alice": "secret123", "bob": "qwerty456"}
    print("--- Problem 2: Login Documentation Formats ---")
```

```
    print(f"Correct password (docstring): {login_docstring('alice', 'secret123', creds)}")
    print(f"Wrong password (inline comments): {login_inline('alice', 'wrongpass', creds)}")
    print(f"Unknown user (Google-style): {login_google('charlie', 'secret123', creds)}")
    print(f"Docstring available: {bool(login_docstring.__doc__)}")
    print(f"Google-style Returns section: {'Returns:' in login_google.__doc__}")
```

Output:



Comparison:



Final Justification

The three approaches make the login function easier to understand, but Google-style docstrings are the most helpful for onboarding because the structured parameter and return sections give new developers a quick reference and can be surfaced by automated documentation tools.

Task Description #3 (Calculator Module - Automatic Documentation Generation)

Task: Design calculator.py with add(a, b), subtract(a, b), multiply(a, b), and divide(a, b). Add appropriate docstrings, display module documentation in the terminal, and generate calculator.html using pydoc.

Prompt:

Create calculator.py with documented arithmetic functions. Use Python documentation tools to display the module documentation in the terminal and export it to HTML with pydoc.

Original Code (Before Documentation):

```
def add(a, b):
    return a + b
```

```
def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    return a / b
```

Documented Revised Version:

```
"""Calculator utility module with documented arithmetic functions."""


def add(a, b):
    """Return the sum of two numbers.

    Args:
        a (int | float): First number.
        b (int | float): Second number.

    Returns:
        int | float: The sum of a and b.
    """
    return a + b


def subtract(a, b):
    """Return the difference between two numbers.

    Args:
        a (int | float): First number.
        b (int | float): Second number.

    Returns:
        int | float: The result of a - b.
    """
    return a - b


def multiply(a, b):
    """Return the product of two numbers.

    Args:
        a (int | float): First number.
        b (int | float): Second number.

    Returns:
        int | float: The product of a and b.
    """
    return a * b


def divide(a, b):
    """Return the quotient of two numbers.

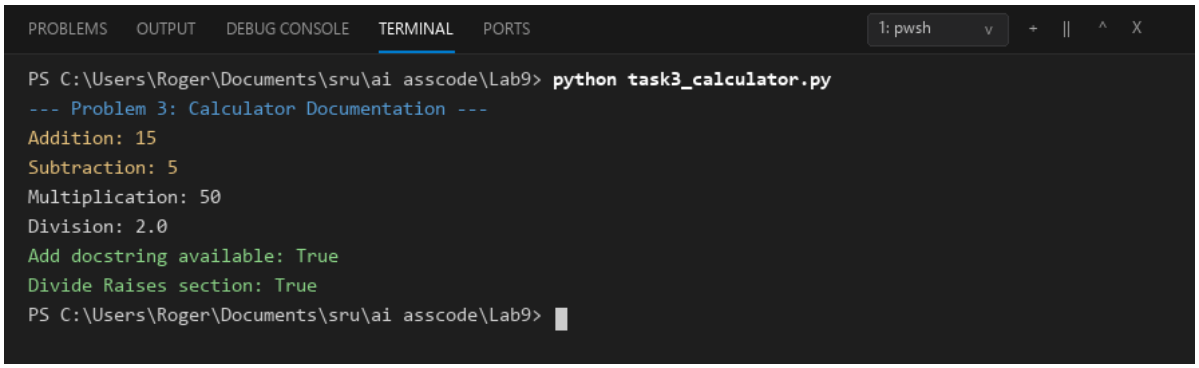
    Args:
        a (int | float): Dividend.
        b (int | float): Divisor.
```

```
    Returns:
        float: The quotient a / b.

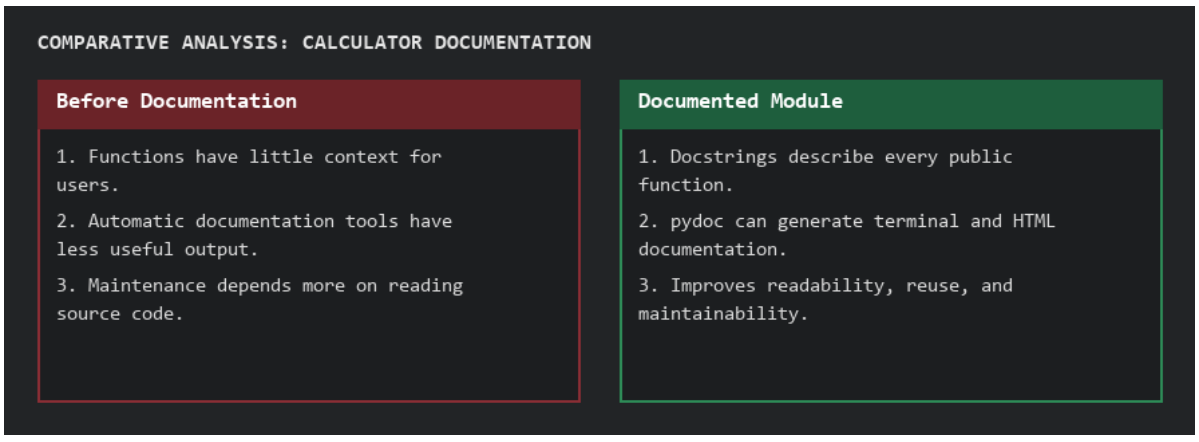
    Raises:
        ZeroDivisionError: If b is zero.
    """
    if b == 0:
        raise ZeroDivisionError("cannot divide by zero")
    return a / b


if __name__ == "__main__":
    print("--- Problem 3: Calculator Documentation ---")
    print(f"Addition: {add(10, 5)}")
    print(f"Subtraction: {subtract(10, 5)}")
    print(f"Multiplication: {multiply(10, 5)}")
    print(f"Division: {divide(10, 5)}")
    print(f"Add docstring available: {bool(add.__doc__)}")
    print(f"Divide Raises section: {'Raises:' in divide.__doc__}")
```

Output:



Comparison:



Final Justification

The calculator module contains docstrings for all required functions. The terminal documentation is generated with Python pydoc, and calculator.html is produced automatically for browser viewing. The documented version improves readability and maintainability.

Task Description #4 (Conversion Utilities Module - Automatic Docs)

Task: Create conversion.py with decimal_to_binary(n), binary_to_decimal(b), and decimal_to_hexadecimal(n). Add docstrings, show terminal documentation, export HTML using pydoc, and open it in a browser.

Prompt:

Create conversion.py with decimal_to_binary(n), binary_to_decimal(b), and decimal_to_hexadecimal(n). Add Google-style docstrings, input validation, and tests for pydoc terminal and HTML export.

Original Code (Before Documentation):

```
def decimal_to_binary(n):
    return format(n, "b")

def binary_to_decimal(b):
    return int(b, 2)

def decimal_to_hexadecimal(n):
    return format(n, "X")
```

Documented Revised Version:

```
"""Conversion utilities module for decimal, binary, and hexadecimal values."""


def decimal_to_binary(n):
    """Convert a non-negative integer to a binary string without '0b'.

    Args:
        n (int): Non-negative decimal integer to convert.

    Returns:
        str: Binary string representation using '0' and '1'.

    Raises:
        TypeError: If n is not an integer.
        ValueError: If n is negative.
    """
    if isinstance(n, bool) or not isinstance(n, int):
        raise TypeError("n must be an integer")
    if n < 0:
        raise ValueError("n must be non-negative")
    return format(n, "b")


def binary_to_decimal(b):
    """Convert a non-empty binary string to a decimal integer.

    Args:
        b (str): Non-empty string containing only '0' and '1'.

    Returns:
        int: Base-10 integer represented by b.

    Raises:
        TypeError: If b is not a string.
        ValueError: If b is empty or contains non-binary characters.
```

```
    """
    if not isinstance(b, str):
        raise TypeError("b must be a string")
    if not b or any(ch not in "01" for ch in b):
        raise ValueError("b must be a non-empty binary string")
    return int(b, 2)


def decimal_to_hexadecimal(n):
    """Convert a non-negative integer to uppercase hexadecimal without '0x'.

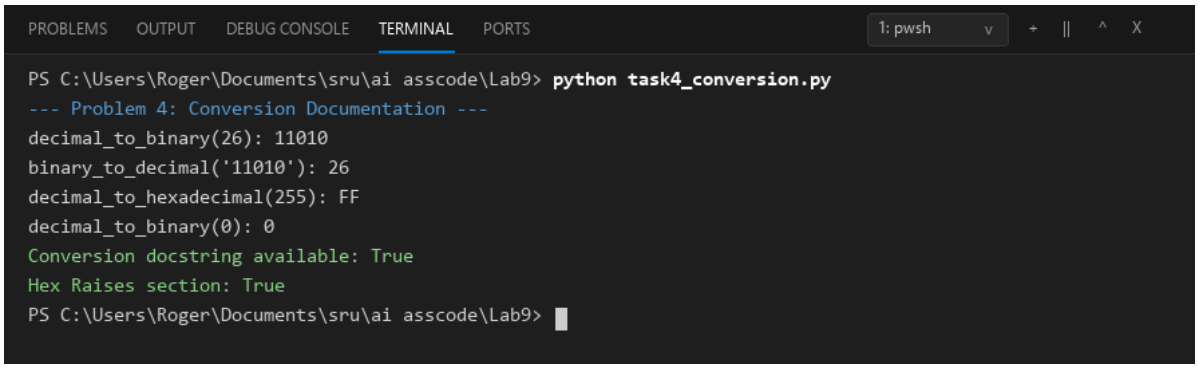
    Args:
        n (int): Non-negative decimal integer to convert.

    Returns:
        str: Uppercase hexadecimal string representation.

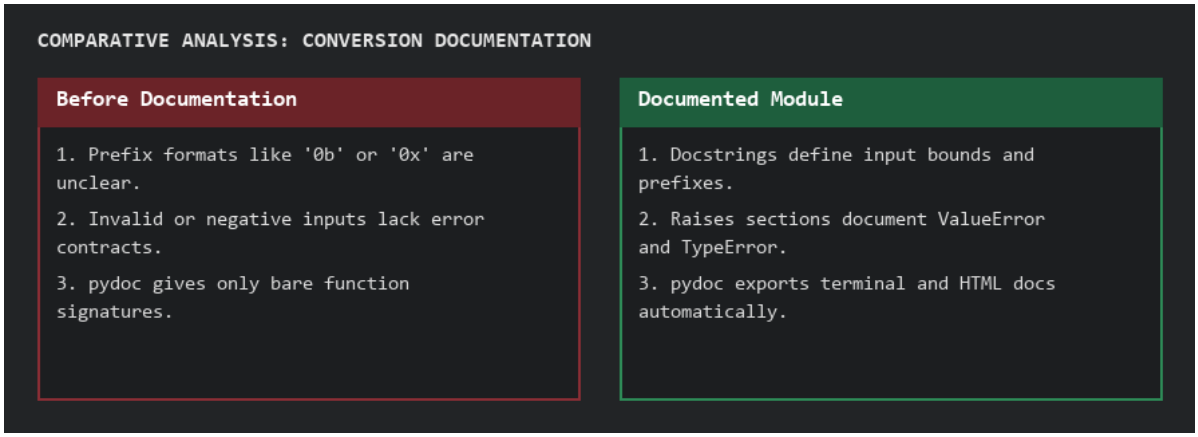
    Raises:
        TypeError: If n is not an integer.
        ValueError: If n is negative.
    """
    if isinstance(n, bool) or not isinstance(n, int):
        raise TypeError("n must be an integer")
    if n < 0:
        raise ValueError("n must be non-negative")
    return format(n, "X")


if __name__ == "__main__":
    print("--- Problem 4: Conversion Documentation ---")
    print(f"decimal_to_binary(26): {decimal_to_binary(26)}")
    print(f"binary_to_decimal('11010'): {binary_to_decimal('11010')}")
    print(f"decimal_to_hexadecimal(255): {decimal_to_hexadecimal(255)}")
    print(f"decimal_to_binary(0): {decimal_to_binary(0)}")
    print(f"Conversion docstring available: {bool(decimal_to_binary.__doc__)}")
    print(f"Hex Raises section: {'Raises:' in decimal_to_hexadecimal.__doc__}")
```

Output:



Comparison:



Final Justification

The conversion module documents accepted types, error conditions, and exact output formatting without 0b or 0x prefixes. Python pydoc uses these Google-style docstrings to generate terminal help and conversion.html automatically.

Task Description #5 (Course Management Module - Automatic Docs)

Task: Create course.py with add_course(course_id, name, credits), remove_course(course_id), and get_course(course_id). Add docstrings, show terminal documentation, export HTML using pydoc, and open it in a browser.

Prompt:

Create course.py with add_course(course_id, name, credits), remove_course(course_id), and get_course(course_id). Add Google-style docstrings, validation checks, and sample tests for pydoc.

Original Code (Before Documentation):

```
courses = {}

def add_course(course_id, name, credits):
    courses[course_id] = {"name": name, "credits": credits}

def remove_course(course_id):
    courses.pop(course_id, None)

def get_course(course_id):
    return courses.get(course_id)
```

Documented Revised Version:

```
"""Course management module with documented in-memory CRUD utilities."""

_courses = {}


def add_course(course_id, name, credits):
    """Add a course to the in-memory course catalogue.

    Args:
```

```
        course_id (str): Unique non-empty course identifier.
        name (str): Descriptive title of the course.
        credits (int): Positive number of credit units.

    Returns:
        dict: Stored course record with course_id, name, and credits.

    Raises:
        TypeError: If course_id/name is not str or credits is not int.
        ValueError: If fields are invalid or course_id already exists.
    """
    if not isinstance(course_id, str) or not isinstance(name, str):
        raise TypeError("course_id and name must be strings")
    if isinstance(credits, bool) or not isinstance(credits, int):
        raise TypeError("credits must be an integer")
    cid = course_id.strip()
    title = name.strip()
    if not cid or not title:
        raise ValueError("course_id and name must not be empty")
    if credits <= 0:
        raise ValueError("credits must be positive")
    if cid in _courses:
        raise ValueError(f"course ID '{cid}' already exists")
    record = {"course_id": cid, "name": title, "credits": credits}
    _courses[cid] = record
    return record


def remove_course(course_id):
    """Remove a course from the catalogue by ID.

    Args:
        course_id (str): Unique identifier of the course to remove.

    Returns:
        bool: True if the course existed and was removed; False otherwise.
    """
    return _courses.pop(course_id, None) is not None


def get_course(course_id):
    """Retrieve a course record by ID.

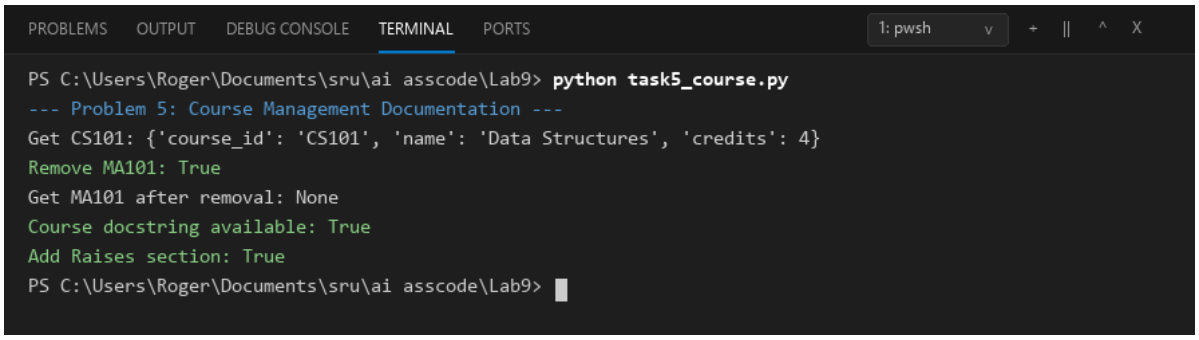
    Args:
        course_id (str): Unique identifier of the course to retrieve.

    Returns:
        dict | None: Course dictionary if found; otherwise None.
    """
    return _courses.get(course_id)

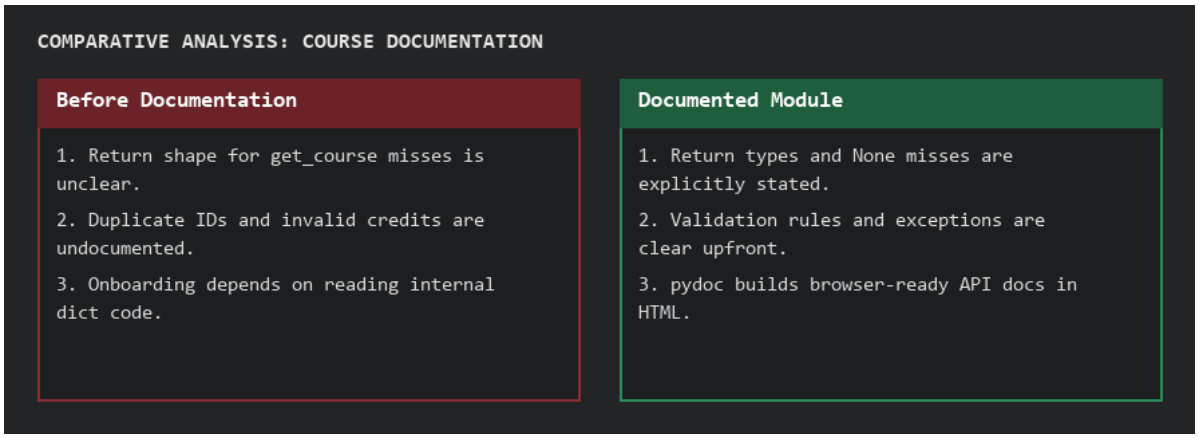

if __name__ == "__main__":
    print("--- Problem 5: Course Management Documentation ---")
    add_course("CS101", "Data Structures", 4)
    add_course("MA101", "Engineering Mathematics", 3)
    print(f"Get CS101: {get_course('CS101')}")
    print(f"Remove MA101: {remove_course('MA101')}")
    print(f"Get MA101 after removal: {get_course('MA101')}")
    print(f"Course docstring available: {bool(add_course.__doc__)}")
```

```
    print(f"Add Raises section: {'Raises:' in add_course.__doc__}")
```

Output:



Comparison:



Final Justification

The course module documents dictionary schemas, duplicate-ID errors, and None/boolean return behavior for lookups and deletions. pydoc turns these structured docstrings into terminal reference pages and course.html documentation.